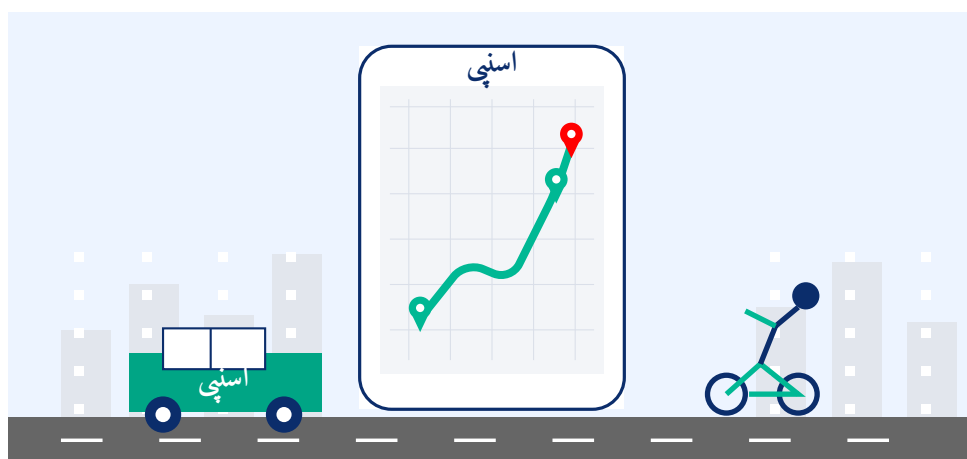


پروژه درس طراحی الگوریتم

سیستم مسیریاب شهری هوشمند



استاد درس: دکتر سپهر
طراح پروژه: پارسا ایمانی

فهرست مطالب

۱	مقدمه و سناریوی پروژه	۲
۲	مدل هزینه مسیر	۲
۱۰۲	جدول جمع‌بندی پارامترها و اثر آن‌ها	۲
۳	ورودی برنامه و ساختار داده پیشنهادی	۳
۴	فاز اول: مدل‌سازی گراف	۴
۵	فاز دوم: مسیریابی پایه با دایجسترا	۴
۱۰۵	شبیه‌کد دایجسترا	۴
۲۰۵	فرمول بازسازی مسیر	۵
۶	فاز سوم: وزن منفی و تشخیص دور منفی	۵
۱۰۶	شبیه‌کد بلن-فورد	۵
۷	فاز چهارم امتیازی: پیک موتوری و مسیر چندمقصودی	۶
۱۰۷	شبیه‌کد نزدیک‌ترین همسایه	۶
۸	قابلیت‌های مورد انتظار برنامه نهایی	۷
۹	خروجی‌های مورد انتظار از دانشجو	۷
۱۰	جدول جمع‌بندی الگوریتم‌ها	۸
۱۱	معیارهای ارزیابی و نمره‌دهی	۸

۱ مقدمه و سناریوی پروژه

فرض کنید به عنوان مهندس الگوریتم در یک شرکت تاکسی اینترنتی شبیه اسنپ استخدام شده‌اید. نخستین وظیفه، طراحی هسته اصلی مسیریابی^۱ برای یک سیستم شهری است. این هسته باید با دریافت داده‌های نقشه، وضعیت مسیرها و شرایط محیطی، مسیر کم‌هزینه‌تر را برای جابه‌جایی مسافریا پیک موتوری پیشنهاد دهد.

در این پروژه، نقشه شهر به صورت یک گراف جهت‌دار و وزن‌دار^۲ مدل می‌شود. گره‌ها^۳ نشان‌دهنده تقاطع‌ها، میدان‌ها یا نقاط مهم شهر هستند. یال‌ها^۴ خیابان‌ها یا مسیرهای بین دو نقطه را نمایش می‌دهند و وزن‌ها^۵ هزینه عبور از هر خیابان را مشخص می‌کنند.

هدف‌های اصلی سیستم

۱. یافتن مناسب‌ترین مسیر بین مبدا و مقصد بر اساس هزینه سفر.
۲. تشخیص شرایط خاص مانند وزن منفی^a و دور منفی^b.
۳. پیشنهاد یک ترتیب مناسب برای پیک موتوری که باید چند مقصد را بازدید کند.

Negative Weight^a
Negative Cycle^b

۲ مدل هزینه مسیر

برای واقعی‌تر کردن مسیریابی، هزینه عبور از یک خیابان فقط برابر طول خیابان نیست. در این مدل، هر یال از گره u به گره v چهار مولفه دارد: مسافت فیزیکی^۶، ضریب ترافیک^۷، ضریب شرایط جاده یا آب‌وهوا^۸ و تاخیر تقاطع مقصد^۹.

فرمول هزینه عبور از خیابان

$$\text{Cost}(u, v) = d(u, v) \times t(u, v) \times w(u, v) + \text{delay}[v]$$

این فرمول به سیستم اجازه می‌دهد مسیر کوتاه از نظر فاصله را با مسیر سریع‌تر از نظر شرایط واقعی مقایسه کند.

۱.۲ جدول جمع‌بندی پارامترها و اثر آن‌ها

Routing Engine^۱
Directed Weighted Graph^۲
Vertices^۳
Edges^۴
Weights^۵
Distance^۶
Traffic Coefficient^۷
Road/Weather Coefficient^۸
Destination Intersection Delay^۹

نماد	معنا	دامنه مقدار	اثر در مساله
d	مسافت خیابان	عدد مثبت	هرچه بزرگ تر باشد، بخش پایه هزینه مسیر بیشتر می شود.
t	ضریب ترافیک	از ۱ تا ۵	مسیر پرترافیک حتی اگر کوتاه باشد، می تواند هزینه بیشتری بگیرد.
w	ضریب شرایط جاده یا آب و هوا	از ۱ تا ۵	شرایط نامساعد مانند لغزندگی یا خرابی مسیر، هزینه عبور را افزایش می دهد.
$\text{delay}[v]$	تاخیر تقاطع مقصد	عدد نامنفی	تاخیر توقف یا کندی حرکت در نقطه ورود به مقصد یال را به هزینه اضافه می کند.

مقادیر ۱ و ۵ در ضرایب ترافیک و شرایط جاده، به ترتیب کمترین و بیشترین شدت را نشان می دهند.^۴

^۴مثلا در ضریب ترافیک، مقدار ۱ یعنی مسیر روان و مقدار ۵ یعنی ترافیک بسیار سنگین. در ضریب شرایط جاده نیز مقدار ۵ می تواند نشان دهنده بارندگی شدید، لغزندگی یا خرابی جدی مسیر باشد.

۳ ورودی برنامه و ساختار داده پیشنهادی

برنامه باید اطلاعات نقشه شهر را از یک فایل متنی مانند map.txt بخواند و سپس آن را در حافظه ذخیره کند. ساختار پیشنهادی برای این کار، فهرست مجاورت^{۱۰} است؛ زیرا در نقشه شهری معمولا تعداد خیابانهای خروجی از هر تقاطع محدود است و این ساختار از نظر حافظه کارآمدتر از ماتریس مجاورت^{۱۱} خواهد بود.

داده های لازم در حافظه

- برای هر گره: شناسه گره و مقدار تاخیر همان گره.
- برای هریال: گره شروع، گره پایان، مسافت، ضریب ترافیک، ضریب آب و هوا و هزینه نهایی محاسبه شده.
- برای بازیابی مسیر: آرایه والد^۴ یا نگاشت والد.
- برای کوتاه ترین فاصله ها: آرایه فاصله^۵.

Parent Array^۴
Distance Array^۵

Adjacency List^{۱۰}
Adjacency Matrix^{۱۱}

۴ فاز اول: مدل سازی گراف

در این فاز، هدف اصلی تبدیل داده خام فایل ورودی به یک گراف قابل پردازش است.

خواسته ها

- خواندن فایل ورودی و استخراج اطلاعات گره ها و یال ها.
- ساخت گراف در حافظه با استفاده از ساختار داده مناسب.
- ذخیره وزن های چندمعیاره، هزینه نهایی یال ها و تاخیر گره ها.
- آماده سازی داده ها برای اجرای الگوریتم های کوتاه ترین مسیر.

تحلیل کوتاه فاز اول: اگر تعداد گره ها $|V|$ و تعداد یال ها $|E|$ باشد، خواندن و ساخت گراف با فهرست مجاورت در زمان $O(|V| + |E|)$ و فضای $O(|V| + |E|)$ انجام می شود.

۵ فاز دوم: مسیریابی پایه با دایجسترا

در حالت عادی، یعنی وقتی همه هزینه های یال ها نامنفی هستند، الگوریتم دایجسترا^{۱۲} انتخاب مناسبی برای پیدا کردن کوتاه ترین مسیر از یک مبدا به مقصد است. برای بهینه سازی، از صف اولویت کمینه^{۱۳} استفاده می شود.

۱.۵ شبهه کد دایجسترا

```
Dijkstra(Graph G, source s):
    for each vertex v in G:
        dist[v] = INF
        parent[v] = NIL
    dist[s] = 0
    PQ = MinHeap()
    PQ.push((0, s))

    while PQ is not empty:
        (du, u) = PQ.popMin()
        if du != dist[u]:
            continue
        for each edge (u, v) in Adj[u]:
            cost = d(u,v) * t(u,v) * w(u,v) + delay[v]
```

^{۱۲} Dijkstra Algorithm
^{۱۳} Min-Heap Priority Queue

```

        if dist[v] > dist[u] + cost:
            dist[v] = dist[u] + cost
            parent[v] = u
            PQ.push((dist[v], v))

    return dist, parent
    
```

۲۰۵ فرمول بازسازی مسیر

پس از پایان الگوریتم، مسیر نهایی با دنبال کردن والد‌ها از مقصد به سمت مبدا ساخته می‌شود:

$$\text{path} = \text{reverse}(\text{target}, \text{parent}[\text{target}], \text{parent}[\text{parent}[\text{target}]], \dots, \text{source})$$

دایجسترا تنها زمانی پاسخ درست تضمین می‌کند که همه وزن‌ها نامنفی باشند.^{۱۴}

^{۱۴} دلیل اصلی این است که دایجسترا پس از تثبیت فاصله یک گره، فرض می‌کند دیگر مسیر ارزان‌تری به آن گره پیدا نخواهد شد. وجود یال منفی می‌تواند این فرض را نقض کند.

پیچیدگی زمانی: با صف اولویت کینه، پیچیدگی زمانی برابر $O((|V|+|E|) \log |V|)$ است. پیچیدگی فضایی: برابر $O(|V|+|E|)$ است.

۶ فاز سوم: وزن منفی و تشخیص دور منفی

اگر در مدل توسعه‌یافته، بعضی یال‌ها هزینه منفی داشته باشند، باید از الگوریتم بلن-فورد^{۱۴} استفاده شود. این الگوریتم می‌تواند کوتاه‌ترین مسیر را در حضور یال منفی پیدا کند و همچنین وجود دور منفی را تشخیص دهد.

۱۰۶ شبه‌کد بلن-فورد

```

BellmanFord(Graph G, source s):
    for each vertex v in G:
        dist[v] = INF
        parent[v] = NIL
    dist[s] = 0

    repeat |V| - 1 times:
        for each edge (u, v) in E:
    
```

```

    if dist[u] != INF and dist[v] > dist[u] + cost(u,v):
        dist[v] = dist[u] + cost(u,v)
        parent[v] = u

    for each edge (u, v) in E:
        if dist[u] != INF and dist[v] > dist[u] + cost(u,v):
            return "NEGATIVE CYCLE FOUND"

    return dist, parent
    
```

خروجی مورد انتظار در این فاز

- اگر دور منفی وجود نداشته باشد، کوتاه‌ترین مسیر و هزینه آن گزارش می‌شود.
- اگر دور منفی وجود داشته باشد، برنامه باید این موضوع را واضح اعلام کند.

در صورت وجود دور منفی، مفهوم کوتاه‌ترین مسیر معتبر نیست.^{۱۴}

^{۱۴} زیرا با تکرار چندباره همان دور، هزینه مسیر می‌تواند بارها کاهش پیدا کند و مقدار حداقلی مشخصی باقی نماند.

پیچیدگی زمانی: $O(|V||E|)$. پیچیدگی فضایی: $O(|V|)$ به‌جز فضای نگهداری گراف.

۷ فاز چهارم امتیازی: پیک موتوری و مسیر چندمقصده

در این فاز، پیک از یک مبدا شروع می‌کند، باید به K مقصد مختلف برود و در پایان دوباره به مبدا بازگردد. این بخش نسخه ساده‌شده‌ای از مسئله فروشنده دوره‌گرد^{۱۵} است. چون این مسئله سخت از نوع ان‌پی^{۱۶} است و تعداد مقصدها کمتر از ۱۰ در نظر گرفته شده، می‌توان از یک روش تقریبی مانند نزدیک‌ترین همسایه^{۱۷} استفاده کرد.

۱۰.۷ شبه‌کد نزدیک‌ترین همسایه

```

NearestNeighbor(start, destinations):
    route = [start]
    current = start
    unvisited = set(destinations)
    totalCost = 0
    
```

Travelling Salesman Problem, TSP^{۱۵}

NP-Hard^{۱۶}

Nearest Neighbor Heuristic^{۱۷}

```

while unvisited is not empty:
    best = NIL
    bestCost = INF
    for each x in unvisited:
        pathCost = ShortestPathCost(current, x)
        if pathCost < bestCost:
            bestCost = pathCost
            best = x
    route.append(best)
    totalCost += bestCost
    current = best
    remove best from unvisited

totalCost += ShortestPathCost(current, start)
route.append(start)
return route, totalCost
    
```

نکته اجرایی: تابع ShortestPathCost می‌تواند بسته به وجود یا نبود وزن منفی، دایجسترا یا بلن-فورد را اجرا کند.^{۱۸}

۸ قابلیت‌های مورد انتظار برنامه نهایی

برنامه نهایی باید بتواند کارهای زیر را انجام دهد:

- خواندن فایل ورودی و ساخت گراف.
- محاسبه هزینه هر یال با استفاده از تابع هزینه تعریف شده.
- پیدا کردن کوتاه‌ترین مسیر با دایجسترا در حالت وزن‌های نامنفی.
- استفاده از بلن-فورد در صورت وجود یال منفی.
- تشخیص و گزارش دور منفی.
- پیشنهاد مسیر تقریبی برای چند مقصد در بخش امتیازی.
- نمایش خوانا و دقیق مسیر، هزینه کل، یال‌های عبوری و پیام‌های خطا.

۹ خروجی‌های مورد انتظار از دانشجو

دانشجو باید پروژه را در قالب یک فایل فشرده تحویل دهد که شامل سورس کد و گزارش کار باشد.

^{۱۸} برای سرعت بهتر، اگر همه وزن‌ها نامنفی باشند، دایجسترا مناسب‌تر است؛ اما در حضور وزن منفی باید از بلن-فورد استفاده شود.

۱. سورس کد برنامه

- کد باید خوانا، منظم و قابل اجرا باشد.
- برای بخش‌های مهم، کامنت مناسب نوشته شود.
- زبان پیاده‌سازی می‌تواند پایتون^a یا سی‌پلاس‌پلاس^b باشد.

Python^a
C++^b

۲. گزارش کار

- توضیح نحوه مدل‌سازی گراف و ساختار داده‌ها.
- توضیح تابع هزینه و تاثیر معیارهای مختلف.
- تحلیل دقیق پیچیدگی زمانی و فضایی.
- توضیح تفاوت کاربرد دایجسترا و بلن-فورد.
- نمایش چند نمونه اجرا و تست کیس.
- بیان فرضیات، محدودیت‌ها و تصمیمات طراحی.

۱۰ جدول جمع‌بندی الگوریتم‌ها

الگوریتم	کاربرد در پروژه	محدودیت مهم	پیچیدگی زمانی رایج
دایجسترا	کوتاه‌ترین مسیر سریع بین مبدا و مقصد	فقط برای وزن‌های نامنفی معتبر است	$O((V + E) \log V)$
بلن-فورد	کوتاه‌ترین مسیر در حضور وزن منفی و تشخیص دور منفی	کندتر از دایجسترا است	$O(V E)$
نزدیک‌ترین همسایه	پیشنهاد مسیر تقریبی برای چند مقصد	همیشه جواب بهینه نمی‌دهد	حدود $O(K^2 \cdot SP)$

در سطر آخر، SP هزینه اجرای الگوریتم کوتاه‌ترین مسیر بین دو نقطه است.^{۱۹}

۱۱ معیارهای ارزیابی و نمره‌دهی

^{۱۹} اگر از دایجسترا استفاده شود، SP برابر پیچیدگی دایجسترا است و اگر از بلن-فورد استفاده شود، برابر پیچیدگی بلن-فورد خواهد بود.

معیار	موارد قابل بررسی
صحت عملکرد الگوریتم‌ها	تولید خروجی درست، پوشش تست کیس‌های متنوع و تشخیص درست حالت‌های خاص.
رعایت تحلیل و محدودیت‌های الگوریتمی	انتخاب ساختار داده مناسب، رعایت پیچیدگی زمانی و فضایی، استفاده صحیح از دایجسترا و بلن-فورد.
کیفیت پیاده‌سازی	خوانایی کد، ساختار مناسب برنامه، مدیریت ورودی و خروجی، مدیریت خطا و پیام‌های واضح.
کیفیت گزارش	توضیحات شفاف، تحلیل نظری درست، ارائه نمونه اجراها و توضیح تصمیمات طراحی.
بخش امتیازی	معقول بودن روش تقریبی برای مسیر چندمقصدی و تحلیل روش انتخاب‌شده.